

CSE4121 — Object Oriented Design and Design Patterns

Tomorrow's Exam: Probable Questions with Complete Answers

Compiled from the teacher's colour-marked sample questions (teal marks), the handwritten suggestion note, CT-1 & CT-2 papers and the Module 1–3 slide decks. 26 compact Q&A — finishable in one day.

Hints honoured: Slide 1 → what is design (conceptual/technical), object types, CRC (responsibility & collaboration); Slide 2 → CT questions; Slide 3 → coupling, cohesion, SoC, conceptual integrity (code may be GIVEN in the question — analyse it, no need to write code in the answer); Behavioural slide → CT-2 questions; pattern slides → GoF; Slide 6 (underlined) → CODE SMELLS, MVC, REFACTORING.

Section A — OO Analysis, Design and Modelling (Slides 1–3 + CT-1)

Q1. Design: what it means; conceptual vs technical design

Source: Teacher suggestion — Slide 1 (“Design ■■■■ ■■ ■■■■■■■■?”, conceptual vs technical)

What is design? Distinguish between conceptual design and technical design.

Design is the activity of planning the structure, behaviour and interactions of a software system before coding — a bridge between analysis (what the system must do) and implementation (how it is coded). It produces models (CRC cards, UML diagrams) instead of code.

Conceptual design

- High-level model made for communication with clients and for understanding the problem.
- Tool: CRC cards — easy communication with clients, focus on design without code.
- Prototype / high-level simulation; no programming details.

Technical design

- Detailed model made for developers as guidance for coding.
- Tool: UML class diagrams — more detailed than CRC cards, easier conversion to code.
- Defines classes, attributes, methods, visibility and relationships.

Conclusion: conceptual design answers “what are the objects and responsibilities?”, technical design answers “how exactly will this be coded?”.

Q2. Object types: entity, control and boundary objects

Source: Teacher suggestion — Slide 1 (“Object type identify”)

How do you identify object types in an object-oriented model? Explain entity, control and boundary objects with examples.

During OO analysis, candidate objects are classified into three stereotypes:

- Entity object: holds persistent data / a real-world thing. DUMS examples: Citizen, Complaint, WasteBin, WasteTruck.
- Control object: coordinates and executes logic between other objects. Examples: ComplaintController, ServiceManager, Scheduler.
- Boundary object: the interface between the system and its actors. Examples: LoginForm, WebDashboard, MobileApp API.

Identification technique

- Nouns in the requirements → entity objects (complaint, citizen, bin).
- Verbs / coordination words (process, manage, schedule) → control objects.
- Interaction points with users or external systems (screen, form, sensor API) → boundary objects.

Example: in DUMS a Citizen (boundary: web form) submits a Complaint (entity); the ComplaintController (control) validates and stores it.

Q3. CRC cards: responsibilities and collaborations

Source: Teacher suggestion — Slide 1 (CRC — Responsibility, Collaboration)

What is a CRC card? Explain responsibility and collaboration with a CRC design of the DUMS smart-city portal.

A CRC card (Class–Responsibility–Collaboration) is an index-card style tool for conceptual design. Each card describes one class:

- Class name (top).
- Responsibilities: what the class must know and do.
- Collaborations: the other classes it must work with to fulfil those responsibilities.

Example card — Citizen

- Responsibilities: store name and address; submit a complaint; track own complaint status.
- Collaborations: Complaint (to create it), ComplaintPortal (boundary used to submit).

Example card — ComplaintManager (control)

- Responsibilities: receive complaints; assign to department; update status.
- Collaborations: Citizen, Department, NotificationService.

CRC cards capture the relationships between objects: a collaboration edge in a CRC session becomes an association in the UML class diagram. They keep the discussion code-free and client-friendly.

Q4. Why UML / problems of coding without a design model (CT-1 Q1)

Source: Slide 2 — CT questions (CT-1 Q1)

A team argues that UML models are unnecessary because modern IDEs can automatically generate code. Evaluate this argument. Or: a developer directly starts coding a Hospital Management System without any design model — analyse the possible problems.

- Misunderstood requirements: without a shared model the developer guesses; wrong features are built and discovered late, when changes are most expensive.
- Poor communication: clients cannot read code; UML is the common visual language between client and developer.
- Tangled, duplicated structure: classes and responsibilities emerge ad hoc → high coupling, low cohesion, code smells.
- No documentation: future maintainers have no blueprint; every change is risky.
- Late error detection: design flaws surface only at testing instead of on paper.

Why the IDE argument fails

IDEs generate code from a model — they cannot create the model itself. Someone must still decide classes, responsibilities and relationships; UML is exactly the notation for making and reviewing those decisions early and cheaply. Evaluation: UML remains necessary for communication, documentation, early validation and maintainable structure.

Q5. Association, aggregation and composition in DUMS (CT-1 Q2)

Source: Slide 2 — CT questions (CT-1 Q2)

DUMS contains citizens, garbage bins, waste collection trucks, collection schedules and monitoring applications. Determine where association, aggregation and composition should be used and justify each choice based on object dependency and lifetime.

Association (weak, independent objects)

- Citizen – Complaint: a citizen files complaints; both exist independently; simple link.

Aggregation (whole–part, part can live without the whole)

- DUMS – WasteTruck / WasteTruck – Driver: the truck and driver can exist and move to another system; shared ownership, independent lifetime.

Composition (strong ownership, same lifetime)

- CollectionSchedule – ScheduleEntry, or Complaint – ComplaintDetails: the part is created and destroyed with the whole; it has no meaning alone.

Justification rule

- Dependency: does the part need the whole to exist? yes → composition, no → aggregation/association.
- Lifetime: destroyed together → composition (filled diamond); outlives → aggregation (hollow diamond); unrelated lifetimes → association (plain line).

Q6. Generalization for reuse: AAS farming machines (CT-1 Q3)

Source: Slide 2 — CT questions (CT-1 Q3)

A smart farming company builds irrigation systems, harvesters and pesticide spraying machines as separate classes without inheritance. Explain the limitations, redesign with a parent–child hierarchy, and show how reusability, maintainability and extensibility improve.

Limitations of independent classes

- Duplicated code: engine on/off, fuel check, GPS logic repeated in every machine class (violates DRY).
- Maintenance cost: a bug in “start engine” must be fixed in three places.
- No polymorphism: a list of “all machines” cannot be handled uniformly.
- Extension pain: every new machine type re-implements everything.

Redesign with generalization

```
abstract class FarmingMachine {
    protected String id; protected double fuel;
    void startEngine() { ... } // common once
    void stopEngine() { ... }
    abstract void doWork(); // varies per child
}
class IrrigationSystem extends FarmingMachine { void doWork() { ... } }
class Harvester extends FarmingMachine { void doWork() { ... } }
class PesticideSprayer extends FarmingMachine { void doWork() { ... } }
```

Improvements

- Reusability: common code written once in the parent.
- Maintainability: one fix point; children stay small.
- Extensibility: a new machine = one new subclass, existing code untouched (OCP).
- Polymorphism: FarmingMachine m = new Harvester(); works in collections.

Section B — Design Complexity, Code Quality (Slide 3)

Q7. Coupling and cohesion

Source: Teacher suggestion — Slide 3 (code may be given in the question; analyse, don't write code)

Define coupling and cohesion. If an exam code snippet shows one class using many other classes, or one class doing unrelated jobs, how do you evaluate it?

Coupling

The degree of dependency between classes. Low (loose) coupling is desired: classes interact through small, stable interfaces instead of each other's internals.

Cohesion

How strongly the responsibilities inside one class belong together. High cohesion is desired: one class, one clear purpose.

Reading the given code

- High-coupling signs: many import/field types of other classes, direct access to another object's private data, long message chains (a.getB().getC().do()).
- Low-cohesion signs: unrelated methods together (login + payment + report in one class), control flags that switch behaviour.

Goal and fixes

Aim: low coupling + high cohesion. Fixes: split classes by responsibility (SRP), hide internals (encapsulation), depend on interfaces (DIP), apply the Law of Demeter.

Q8. Separation of Concerns and conceptual integrity

Source: Teacher suggestion — Slide 3

Explain Separation of Concerns (SoC) with the SmartPhone example and define conceptual integrity.

Separation of Concerns

- Initial design: one SmartPhone class containing both camera and phone functions → low cohesion, poor modularity, components cannot be reused or replaced separately.
- Improved design: identify two concerns — phone functionality and camera functionality; create separate cohesive classes Phone and Camera; SmartPhone acts as coordinator referencing them, providing access without needing internal details.

Effect: each concern can be changed, tested and reused independently (e.g. upgrade the camera without touching call logic).

Conceptual integrity

The design follows one consistent philosophy and set of conventions — same naming style, one error-handling strategy, one way of layering — so the whole system looks as if a single mind designed it. It rejects ad-hoc additions that break consistency; together with SoC it keeps the architecture understandable and maintainable.

Q9. DRY, Principle of Least Knowledge, OCP, Information Hiding

Source: Teacher colour mark — Slide 6 (Remember 1 / Understand 2)

What are the DRY principle, Principle of Least Knowledge, Open/Closed Principle and Information Hiding? How does DRY improve maintainability?

- DRY (Don't Repeat Yourself): every piece of knowledge has a single, unambiguous implementation. Duplicated validation in five classes violates DRY; extracting one Validator means one fix point → fewer inconsistencies, easier change → better maintainability.
- Principle of Least Knowledge (Law of Demeter): a class should talk only to its direct friends; avoid chains like `citizen.getComplaint().getDepartment().getManager()`. Less coupling, easier modification.
- Open/Closed Principle (OCP): software entities should be open for extension but closed for modification — add behaviour via new subclasses/polymorphism, not by editing working code (e.g. replace a large switch with polymorphism).
- Information Hiding: internal details are private behind a public interface (black box); clients depend only on what is exposed, so internals can change safely.

Section C — Code Smells, Anti-patterns and Refactoring (Slide 6 — underlined priority)

Q10. Anti-pattern, code smell; Large Class, Long Method, Duplicated Code, Primitive Obsession

Source: Teacher colour marks — Slide 6 + underlined “code smell”

Define an anti-pattern and a code smell and give three examples. Define Large Class, Long Method, Duplicated Code and Primitive Obsession.

Definitions

- Anti-pattern: a commonly used but ineffective design practice that leads to poor software (e.g. God Class, Golden Hammer).
- Code smell: a surface symptom in the code that indicates a deeper design problem; not a bug, but a maintainability risk.

The four smells

- Large Class: one class carrying many responsibilities (e.g. CityManagement handling traffic + waste + payment + login) → low cohesion, hard to understand and change.
- Long Method: a method doing too many steps (validation + DB + calculation + reporting) → hard to read, test and reuse.
- Duplicated Code: the same logic copied in several places → a bug must be fixed everywhere; copies drift apart.
- Primitive Obsession: using raw types (String “23.8103,90.4125”, int money) instead of meaningful objects (Location, Money) → no validation, no behaviour.

Three quick examples to quote: (1) a 300-line ServiceManager class, (2) identical student-validation code in five classes, (3) switch(service==“Traffic”/“Waste”) repeated in five methods.

Q11. 100-line method: identify smell and refactor

Source: Teacher colour marks — Slide 6 (Apply 4 / Analyze 5)

A method contains 100 lines performing validation, database access, calculations and report generation. Identify the likely code smell and suggest an appropriate refactoring approach. (Same for a 150-line variant.)

Smells present

- Long Method: one method with four unrelated phases.
- Low cohesion / possible Large Class if the owner class holds several such methods.
- Violates Single Responsibility and DRY (logic cannot be reused separately).

Refactoring approach

- Extract Method: split into validate(), saveToDatabase(), calculate(), generateReport(); the original method becomes a short, readable sequence of named steps.
- Give each extracted method a single responsibility and an intention-revealing name.
- Move methods to other classes if responsibilities belong elsewhere (Extract Class).
- Afterwards each piece is independently testable and reusable.

Effect on maintainability: reading time drops, bugs localise to one small method, and adding a new report type touches only generateReport().

Q12. Duplicated switch statement / duplicated validation

Source: Teacher colour marks — Slide 6 (Analyze 5 / Apply 4)

The same switch statement for customer types appears in five different methods (or: the same student-validation code in five classes). Identify the code smell and explain why it creates maintenance problems; propose a refactoring.

Smells

- Duplicated Code (five copies of the same logic).
- Large Switch Statements: repeated conditional logic indicates a missing abstraction.

Why maintenance suffers

- A change (new customer type / new validation rule) must be repeated in five places; missing one copy creates inconsistent behaviour.
- Bug fixes multiply; copies drift apart over time (DRY violation).
- Every switch site violates OCP — existing code must be modified to extend.

Refactoring

- Short term: Extract Method — one shared validate(Student s) used everywhere.
- Better: Replace Conditional with Polymorphism — CustomerType hierarchy (Regular, VIP, Corporate), each subclass implements its own behaviour; the switch disappears and new types are added without editing old code (OCP).

Q13. Are comments always beneficial? Comments as a code smell

Source: Teacher colour mark — Slide 6 (Evaluate 4)

Evaluate whether comments are always beneficial in software development. Explain how excessive comments may sometimes indicate a design problem.

No — comments are not always beneficial.

- Good comments: explain WHY (constraints, legal rules, non-obvious trade-offs), document APIs.
- Bad comments: explain WHAT the code does line by line; they compensate for unclear code and rot when the code changes (misleading comments are worse than none).

Why excessive comments signal a design problem: a long comment block above a 40-line chunk usually means that chunk is a separate responsibility — the correct fix is Extract Method with a good name, after which the comment is unnecessary. Many “how it works” comments in one class often indicate a Large Class / Long Method smell. Evaluation: prefer self-documenting code (small methods, intention-revealing names); keep comments only for intent the code cannot express.

Q14. Refactoring plan for Large Class + Long Methods + Duplicated Code; importance of context

Source: Teacher colour marks — Slide 6 (Create 7 / Evaluate 6)

Create a refactoring plan for a Java application suffering from a Large Class, Long Methods and Duplicated Code, explaining the sequence of improvements. Also evaluate: “A large class is always an example of bad design.”

Sequence of improvements

- 1. Add/identify tests so behaviour stays unchanged (refactoring must not change external behaviour).
- 2. Extract Method on the long methods → small named steps.
- 3. Remove duplication: Extract Method + move shared code to one place (or Pull Up to a superclass).
- 4. Extract Class / Move Method: split the Large Class by responsibility (SRP), e.g. CityManagement → TrafficService + WasteService + EmergencyService.
- 5. Replace remaining switches with polymorphism (OCP).
- 6. Re-run tests after each small step; commit after each step.

Evaluating the statement

Not always. Size alone is not the smell — cohesion is. A large but single-purpose class (e.g. a cohesive builder or a data-heavy configuration holder with one reason to change) can be acceptable. A class is bad when it mixes unrelated responsibilities (many reasons to change). Context — purpose, cohesion, change frequency — must be considered before declaring a smell or refactoring.

Section D — MVC (Slide 6 — underlined priority)

Q15. MVC: components, responsibilities, separation of concerns

Source: Teacher colour marks — Slide 6 (all MVC items marked)

What are the three components of the MVC Pattern? Explain their responsibilities and how MVC achieves separation of concerns.

Components

- Model: application data and business rules (Student, Course, validation, seat counts). Knows nothing about screens.
- View: presentation — renders model data as pages/forms (HTML). Contains no business rules.
- Controller: receives user input (HTTP requests), calls the model to update/query data, selects the view.

Request flow (course-registration example)

(1) student submits form → (2) RegistrationController validates and calls Model.register() → (3) controller chooses the confirmation View → (4) view reads model data → (5) page returned to browser.

How MVC gives SoC

- Each concern lives in its own component: business rules (model), presentation (view), input flow (controller).
- Changing the UI never touches business rules; one model can serve multiple views; each part is independently testable → maintainable and extensible.

Q16. Apply MVC to a university student registration system

Source: Teacher colour marks — Slide 6 (Apply 4 / Create 7)

Apply MVC to a university student registration system (or design an MVC-based Student Management System). Identify the major classes and how they communicate.

- Model classes: Student (id, name), Course (code, seats), Registration (rules: prerequisites, seat availability; methods register(), drop()).
- View classes: RegistrationFormView (HTML form), ResultView (success/failure page), CourseListView.
- Controller: RegistrationController — handles the POST, calls Registration.register(student, course), then selects ResultView.

Communication

Browser → Controller (request). Controller → Model (invoke business operation). Controller → View (choose). View → Model (read data to render). The Model never talks to the View directly — the separation keeps every change local: new page design = new View only; new rule (e.g. credit limit) = Model only.

Section E — Creational and Structural Patterns (Module 1, colour-marked)

Q17. Design pattern, GoF catalog, three categories, the seven Module-1 patterns

Source: Teacher colour marks — Slide 4/Module-1 sample questions

What is a design pattern? Name the three major categories of GoF patterns and list the seven patterns covered in Module 1. Why are patterns “reusable solutions, not ready-made programs”?

A design pattern is a practical, proven, reusable solution to a recurring software design problem — it guides structure and improves flexibility, reusability and maintainability. It is a design template you adapt, not a code library you copy.

- GoF catalog: the “Gang of Four” (Gamma, Helm, Johnson, Vlissides) book with 23 classic OO patterns.
- Creational — control object creation: Singleton, Factory Method.
- Structural — compose classes/objects into larger structures: Adapter, Facade, Composite, Proxy, Decorator.
- Behavioural — object communication: Observer, Command, Chain of Responsibility, State, Template Method.

Module 1’s seven: Singleton, Factory Method (creational) + Adapter, Facade, Composite, Proxy, Decorator (structural). Not ready-made programs because a pattern describes relationships and responsibilities; you must still write the concrete classes for your problem.

Q18. Singleton + one shared configuration object (Java)

Source: Teacher colour marks — Slide 4 (Remember/Apply Singleton items)

Define the Singleton Pattern. An application requires exactly one shared configuration object throughout the system — which pattern and how would you implement it?

Singleton ensures a class has exactly one instance and provides a global access point. It fits application-wide configuration, a shared logger, a database gateway.

```
public class AppConfig {  
    private static AppConfig instance;    // the single instance  
    private String dbName;               // configuration data  
    private AppConfig() {}               // private constructor: no new AppConfig()  
    public static AppConfig getInstance() {  
        if (instance == null)  
            instance = new AppConfig();  
        return instance;  
    }  
    public String getDbName() { return dbName; }  
}
```

- Private constructor blocks external new.
- Static getInstance() creates lazily and returns the same object forever.
- Every module calls AppConfig.getInstance() → one shared object.

Evaluation note: using Singleton for EVERY shared object is over-use — it creates hidden global state and makes testing hard; use it only when exactly-one-instance is a true requirement.

Q19. Factory Method: university course creation

Source: Teacher colour marks — Slide 4 (Create 7 Factory Method)

Design a Factory Method-based system that creates TheoryCourse, LabCourse and ProjectCourse. Describe the classes and their relationships.

Factory Method defines an interface for creating an object but lets subclasses decide which class to instantiate — object creation is separated from object usage.

```
abstract class Course { abstract String type(); }
class TheoryCourse extends Course { String type() { return "Theory"; } }
class LabCourse extends Course { String type() { return "Lab"; } }
class ProjectCourse extends Course { String type() { return "Project"; } }

abstract class CourseFactory { abstract Course createCourse(); } // factory method
class TheoryFactory extends CourseFactory { Course createCourse() { return new TheoryCourse(); } }
class LabFactory extends CourseFactory { Course createCourse() { return new LabCourse(); } }
class ProjectFactory extends CourseFactory { Course createCourse() { return new ProjectCourse(); } }
```

Client code works only with CourseFactory and Course — no new statements in the client. Adding a new course type means a new subclass pair, existing code untouched (OCP). Relationships: Creator hierarchy parallel to Product hierarchy (class factory pattern).

Q20. Adapter: incompatible interfaces (payment + IoT sensor)

Source: Teacher colour marks — Slide 4 + CT-2 Q3

A legacy payment system provides makePayment() while a new application expects pay(). Which pattern and how? Also state what problem the Adapter solves, with the AAS IoT soil-sensor example.

Problem it solves

Two existing classes cannot work together because their interfaces differ. Adapter converts the interface of a class into the interface the client expects, without modifying either existing class.

```
interface PaymentService { void pay(double amount); } // target
class LegacyPayment { public void makePayment(double a) { ... } } // adaptee
class PaymentAdapter implements PaymentService {
    private LegacyPayment legacy;
    PaymentAdapter(LegacyPayment l) { this.legacy = l; }
    public void pay(double amount) { legacy.makePayment(amount); } // translate
}
// client: PaymentService ps = new PaymentAdapter(new LegacyPayment()); ps.pay(500);
```

AAS example

The existing irrigation controller accepts a standard soil-moisture sensor; a new IoT sensor has a different interface. An IoT SensorAdapter implements the standard sensor interface and internally translates the IoT readings — the controller is reused unchanged. UML: Client → Target interface; Adapter implements Target and associates the Adaptee.

Q21. Facade vs Adapter (+ Proxy for access control)

Source: Teacher colour marks — Slide 4 (Evaluate Facade/Adapter, Proxy definition)

Explain the difference between the Facade and Adapter patterns and evaluate which is appropriate when two existing classes have incompatible interfaces. When would a Proxy be more suitable?

- Adapter: converts one interface into another so incompatible classes can collaborate; used when the functionality is right but the interface is wrong.
- Facade: provides a new, simple, unified interface over a whole complex subsystem (e.g. HomeTheaterFacade.startMovie() hides amplifier + projector + lights calls); used to reduce client complexity.
- Proxy: same interface as the real subject but controls access (protection, remote, lazy loading — e.g. load a huge image only when displayed).

Evaluation

Incompatible existing interfaces → Adapter (it is precisely the converter). Facade would be wrong: it does not convert, it simplifies and adds a new interface. For controlling access to an expensive remote service, Proxy (caching + access control) beats Facade, which only simplifies calls.

Q22. Composite for File/Folder hierarchy

Source: Teacher colour mark — Slide 4 (Analyze 5 Composite)

Analyze a file-system design containing File and Folder objects. Explain why the Composite Pattern is appropriate for representing this hierarchy.

Composite composes objects into tree hierarchies (part-whole) and lets clients treat individual objects and compositions uniformly.

- Common component: interface `FileSystemComponent { String getName(); long getSize(); }`.
- Leaf: File — returns its own size.
- Composite: Folder — holds a list of children (Files and Folders); `getSize()` sums children recursively.

Why appropriate: the file system is inherently a tree; clients can call `getSize()/print()` on ANY node without checking “is this a file or a folder?” — uniform treatment, easy recursion, and adding new node types (e.g. Shortcut) needs no client change (OCP). The same structure models City → Zone → Ward → Bin in DUMS.

Q23. Decorator: coffee with milk, sugar and whipped cream

Source: Teacher colour marks — Slide 4 (Understand/Apply Decorator)

A coffee application must dynamically add milk, sugar and whipped cream to a basic coffee object. Which pattern? Explain the implementation and why it beats subclassing every combination.

Decorator — attaches additional responsibilities to an object dynamically by wrapping it, as an alternative to subclassing.

```
interface Beverage { String desc(); double cost(); }
class BasicCoffee implements Beverage { ... cost 50 ... }
abstract class CondimentDecorator implements Beverage {
    protected Beverage inner; // wraps the component
}
class Milk extends CondimentDecorator { double cost() { return inner.cost() + 10; } }
class Sugar extends CondimentDecorator { double cost() { return inner.cost() + 5; } }
class Cream extends CondimentDecorator { double cost() { return inner.cost() + 15; } }
// Beverage b = new Cream(new Sugar(new Milk(new BasicCoffee())));
```

- Subclassing every combination (CoffeeWithMilk, CoffeeWithMilkAndSugar, ...) explodes: 2^n classes — class explosion smell.
- Decorators combine at runtime in any order, add/remove responsibilities dynamically, and each decorator has one responsibility (SRP, OCP).

Same idea in DUMS: basic Notification decorated with EmailDecorator, SMSDecorator, PushDecorator.

Section F — Behavioural Patterns (Module 2 / CT-2)

Q24. Observer: DUMS complaint status notifications (CT-2 Q1)

Source: Behavioural slide — CT-2 Q1

The DUMS complaint system must automatically notify the citizen, the department officer and the city administrator whenever a complaint status changes. Implement with an appropriate pattern.

Observer — a one-to-many dependency: when the Subject changes state, all registered Observers are notified automatically.

```
interface Observer { void update(String status); }
class Citizen implements Observer { ... }
class DepartmentOfficer implements Observer { ... }
class CityAdministrator implements Observer { ... }
class Complaint { // subject
    private String status;
    private List<Observer> observers = new ArrayList<>();
    void attach(Observer o) { observers.add(o); }
    void setStatus(String s) { status = s;
        for (Observer o : observers) o.update(s); } // notify all
}
```

Subject and observers are loosely coupled: adding a new user type (e.g. Mayor) needs only a new Observer class — Complaint is untouched (OCP).

Q25. Chain of Responsibility: university approval and bank cheque

Source: Behavioural slide — CT-2 Q2 + slide sample

Apply the Chain of Responsibility Pattern to the approval chain Teacher → Chairman → Dean → Vice Chancellor. Also explain the bank cheque authorisation example.

Chain of Responsibility passes a request along a chain of handlers until one handler can process it — sender and receiver are decoupled.

```
abstract class Approver {
    protected Approver next;
    void setNext(Approver n) { next = n; }
    abstract void handle(Request r);
}
class Teacher extends Approver { void handle(Request r) {
    if (r.level <= 1) approve(r); else next.handle(r); } }
// Chairman (level 2), Dean (level 3), ViceChancellor (level 4) same idea
```

Bank cheque example

- Amount < Tk 10,000 → Cashier authorises.
- Tk 10,000–10,00,000 → Cashier + Senior Officer.
- > Tk 10,00,000 → Senior Officer + Manager.
- Each handler checks its authority and forwards otherwise; new approval levels can be inserted without changing the client.

Q26. Command and State: remote control and order states (+ pattern drill)

Source: Behavioural slide — Module-2 sample questions (Create/Analyze)

(a) Design a remote control using the Command Pattern (LightOn, LightOff, FanOn). (b) How does the State Pattern improve a design where a large switch controls behaviour by state?

(a) Command Pattern

Encapsulates a request as an object, decoupling the invoker (button) from the receiver (device); supports undo, queueing, logging.

```
interface Command { void execute(); }
class Light { void on(){} void off(){} } // receiver
class LightOnCommand implements Command {
    Light l; LightOnCommand(Light l){this.l=l;}
    public void execute(){ l.on(); }
}
class Remote { // invoker
    Command slot;
```

```
void setCommand(Command c){ slot = c; }  
void pressButton(){ slot.execute(); }  
}
```

(b) State Pattern

When an object's behaviour depends on its state and a giant switch(orderStatus) repeats everywhere, State moves each state into its own class (Pending, Paid, Shipped, Delivered implementing OrderState); the context delegates to the current state object. Adding a state = new class, no switch editing (OCP); the switch smell disappears.

Quick identification drill

- One shared object → Singleton; creation by subclass → Factory Method; incompatible interfaces → Adapter; simplify subsystem → Facade; tree hierarchy → Composite; access control → Proxy; add features dynamically → Decorator; notify subscribers → Observer; approval levels → Chain of Responsibility; request as object/undo → Command; behaviour by state → State; fixed algorithm skeleton → Template Method.